

How to Add New Pages to our Go + HTMX Application

Contents

1. Overview	4
1.1. What We're Building	4
1.2. Prerequisites	4
2. Architecture Recap	5
3. Step 1: Add SQL Query	6
3.1. Update queries.sql	6
3.2. Verify SQL Syntax	6
4. Step 2: Generate SQLC Code	7
4.1. Run sqlc Generate	7
4.2. What Gets Generated	7
5. Step 3: Add Service Method	8
5.1. Update user_service.go	8
5.2. Add Dedicated Mapping Function	8
5.3. Explanation	9
6. Step 4: Add Web Handler Method	10
6.1. Update web_handler.go	10
6.2. Key Features	12
7. Step 5: Create New Template	13
7.1. Create user_list_by_id.html	13

7.2. Template Features	17
8. Step 6: Add Route	19
8.1. Update main.go	19
8.2. Route Pattern	19
8.3. Route Ordering Considerations	19
9. Step 7: Update base.html for Dynamic HTMX Target	21
9.1. The Problem	21
9.2. The Fix: Pass listEndpoint from the handler	21
9.3. View Toggle in base.html	23
10. Step 8: Test the New Page	24
10.1. Build and Start	24
10.2. Test the Page	24
10.3. Test Pagination	24
10.4. Test HTMX Interactions	25
10.5. Verify the Endpoint	25
11. Complete File List	26
12. Troubleshooting	27
12.1. Common Issues	27
12.2. Debug Commands	27
12.3. Logging	28
13. Best Practices	29
13.1. When Adding New Pages	29
13.2. Naming Conventions	29
13.3. Performance Considerations	29
14. Conclusion	30
15. Additional Resources	31

16. Appendix A: Summary of Corrections from v13	31
---	----

patrick toca v14.0, September 10th, 2026

Start Date: September 6th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Overview

This document describes the complete process of adding a new page to a Go + Gin + HTMX + Tailwind CSS + PostgreSQL application using sqlc for database operations.

1.1. What We're Building

We'll add a new page that displays users sorted by ID instead of the default sorting by creation date.

1.2. Prerequisites

- Go 1.21+
- PostgreSQL with **MyWebApp** database
- Node.js and npm installed
- sqlc installed and configured
- Existing project structure with:
 - Gin web framework
 - HTMX for dynamic interactions
 - Tailwind CSS for styling
 - sqlc for type-safe database queries

2. Architecture Recap

Before diving in, it helps to recall the layered architecture of the application:

Table 1. System Architecture

Layer	Component	Responsibility
SQL	<code>queries.sql</code>	Define named queries with sqlc annotations
Repository	<code>internal/repository/*.go</code>	sqlc-generated, type-safe DB access
Service	<code>internal/service/user_service.go</code>	Business logic, domain mapping
Handler	<code>internal/web/web_handler.go</code>	HTTP request handling, HTMX detection
Template	<code>internal/templates/*.html</code>	HTML rendering, HTMX attributes
Routing	<code>cmd/api/main.go</code>	Route registration

Data flows top-to-bottom on a request, and bottom-to-top on a response. Each layer has a single responsibility, and new pages should follow this pattern strictly.

3. Step 1: Add SQL Query

The first step is to add the new query to your `queries.sql` file.

3.1. Update `queries.sql`

Open `queries.sql` and add the following query:

```
-- name: ListUsersByID :many
SELECT id, email, username, full_name, is_active, role, last_login, created_at, updated_at
FROM users
ORDER BY id ASC
LIMIT $1 OFFSET $2;
```



The `:many` directive tells sqlc this query returns multiple rows. The `-- name:` annotation is required for sqlc to generate the method.

3.2. Verify SQL Syntax

Ensure your query is valid by testing it directly in PostgreSQL:

```
psql -d MyWebApp -c "SELECT id, email, username, full_name, is_active, role, last_login, created_at, updated_at FROM users ORDER BY id ASC LIMIT 10 OFFSET 0;"
```

4. Step 2: Generate SQLC Code

After adding the query, regenerate the sqlc code.

4.1. Run sqlc Generate

```
make generate
```

Or manually:

```
sqlc generate
```

4.2. What Gets Generated

sqlc will create:

- `ListUsersByID` method in `internal/repository/queries.sql.go`
- `ListUsersByIDParams` struct
- `ListUsersByIDRow` struct
- Updated `Querier` interface in `internal/repository/querier.go`



If the generation fails, check for SQL syntax errors in `queries.sql`. Common issues include missing semicolons or incorrect table/column names.

5. Step 3: Add Service Method

Add the new service method to handle business logic.

5.1. Update `user_service.go`

Open `internal/service/user_service.go` and add:

```
func (s *UserService) ListUsersByID(ctx context.Context, limit, offset int32) ([]domain.User, error) {
    users, err := s.queries.ListUsersByID(ctx, repository.ListUsersByIDParams{
        Limit:  limit,
        Offset: offset,
    })
    if err != nil {
        return nil, err
    }

    result := make([]domain.User, len(users))
    for i, user := range users {
        result[i] = *s.mapListUsersByIDRowToDomain(&user)
    }
    return result, nil
}
```

5.2. Add Dedicated Mapping Function



Unlike what you might expect, you cannot reuse `mapListUsersRowToDomain` here. That function accepts `*repository.ListUsersRow`, but `ListUsersByID` returns `*repository.ListUsersByIDRow`. These are distinct Go types generated by sqlc, even though they are structurally identical. You must add a dedicated mapper.

Add this function alongside the other mapping functions in `user_service.go`:

```
func (s *UserService) mapListUsersByIDRowToDomain(row *repository.ListUsersByIDRow) *domain.User {
    return &domain.User{
        ID:      row.ID,
        Email:   row.Email,
        Username: row.Username,
        FullName: row.FullName.String,
        IsActive: row.IsActive.Bool,
        Role:    row.Role.String,
        LastLogin: timestampzToUTCtimePtr(row.LastLogin),
        CreatedAt: domain.UTCTime(timestampzToUTCtime(row.CreatedAt)),
        UpdatedAt: domain.UTCTime(timestampzToUTCtime(row.UpdatedAt)),
    }
}
```

5.3. Explanation

- `ctx context.Context`: For request lifecycle management
- `limit, offset int32`: Pagination parameters
- Returns: `[]domain.User` and error
- Uses `mapListUsersByIDRowToDomain` for consistent mapping into the domain layer

6. Step 4: Add Web Handler Method

Add the new handler method to serve the page.

6.1. Update web_handler.go

Open `internal/web/web_handler.go` and add:

```
// ListUsersByIDPage renders the user list sorted by ID
func (h *WebHandler) ListUsersByIDPage(c *gin.Context) {
    log.Println(" [ListUsersByIDPage] Called")

    page := GetPage(c)
    limit := GetLimit(c)
    offset := (page - 1) * limit

    log.Printf(" [ListUsersByIDPage] Params: page=%d, limit=%d", page, limit)

    users, err := h.service.ListUsersByID(c.Request.Context(), int32(limit), int32(offset))
    if err != nil {
        log.Printf(" [ListUsersByIDPage] ListUsersByID error: %v", err)
        c.HTML(http.StatusInternalServerError, "base.html", gin.H{
            "title": "Error",
            "error": err.Error(),
        })
        return
    }

    total, err := h.service.CountUsers(c.Request.Context())
    if err != nil {
        log.Printf(" [ListUsersByIDPage] CountUsers error: %v", err)
        total = int64(len(users))
    }
}
```

```

}

log.Printf("[ListUsersByIDPage] Retrieved %d users, total: %d", len(users), total)

viewUsers := make([]UserViewModel, len(users))
for i, u := range users {
    viewUsers[i] = ToUserViewModel(&u)
}

isHTMX := IsHTMX(c)
log.Printf("[ListUsersByIDPage] IsHTMX: %v", isHTMX)

if isHTMX {
    log.Println("[ListUsersByIDPage] Rendering HTMX partial: user_list_by_id.html")
    c.HTML(http.StatusOK, "user_list_by_id.html", gin.H{
        "users":      viewUsers,
        "page":        page,
        "limit":       limit,
        "total":       total,
        "totalPages": (total + int64(limit) - 1) / int64(limit),
        "title":       "Users Ordered by ID",
        "orderBy":     "ID",
    })
    return
}

log.Println("[ListUsersByIDPage] Rendering full page: base.html")
c.HTML(http.StatusOK, "base.html", gin.H{
    "title":       "Users by ID",
    "users":       viewUsers,
    "page":        page,
    "limit":       limit,
    "total":       total,
    "totalPages": (total + int64(limit) - 1) / int64(limit),

```

```
        "orderBy": "ID",  
        "listEndpoint": "/web/users/by-id",  
    })  
}
```

6.2. Key Features

- `GetPage(c)` and `GetLimit(c)`: Helper functions for pagination
- HTMX detection: `IsHTMX(c)` determines if a partial or full page should be rendered
- Consistent error handling with user-friendly messages
- `listEndpoint` is passed to `base.html` so the initial HTMX load fetches the correct URL

7. Step 5: Create New Template

Create a new partial template for the ID-sorted list.

7.1. Create `user_list_by_id.html`

Create `internal/templates/partials/user_list_by_id.html`:

```
<div class="fade-in">
  <!-- Header -->
  <div class="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 mb-6">
    <div>
      <h2 class="text-2xl font-bold text-gray-900">Users by ID</h2>
      <p id="user-stats" class="text-sm text-gray-600 mt-1">
        {{.total}} total users ▯ sorted by ID
      </p>
    </div>
    <div class="flex flex-wrap gap-3">
      <!-- View Toggle -->
      <div class="flex gap-1 bg-gray-100 rounded-lg p-1">
        <a href="/web/users"
          class="px-3 py-1 text-sm rounded-md transition text-gray-600 hover:text-gray-900">
          By Date
        </a>
        <a href="/web/users/by-id"
          class="px-3 py-1 text-sm rounded-md transition bg-white shadow text-gray-900">
          By ID
        </a>
      </div>

      <!-- Add User Button -->
      <button class="bg-blue-600 text-white px-4 py-2 rounded-lg hover:bg-blue-700 transition flex items-center gap-2">
```

```

        hx-get="/web/users/new"
        hx-target="#user-form-container"
        hx-swap="innerHTML">
<svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
  <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 4v16m8-8H4"/>
</svg>
Add User
</button>
</div>
</div>

<!-- User Form Container -->
<div id="user-form-container" class="mb-6"></div>

<!-- Users Table -->
<div class="bg-white rounded-lg shadow overflow-hidden">
  <div class="overflow-x-auto">
    <table class="min-w-full divide-y divide-gray-200">
      <thead class="bg-gray-50">
        <tr>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
            ID <span class="text-blue-600">🔗</span>
          </th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Username</th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Full Name</th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Email</th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Role</th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Status</th>
          <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">Actions</th>
        </tr>
      </thead>
      <tbody id="user-tbody" class="bg-white divide-y divide-gray-200">
        {{range .users}}
          {{template "user_row.html" .}}
        {{end}}
      </tbody>
    </table>
  </div>
</div>

```

```

        {{else}}
        <tr>
            <td colspan="7" class="px-6 py-12 text-center text-gray-500">
                <p class="mt-2">No users found</p>
            </td>
        </tr>
        {{end}}
    </tbody>
</table>
</div>

<!-- Pagination -->
{{if gt .totalPages 1}}
<div class="bg-white px-4 py-3 border-t border-gray-200 sm:px-6">
    <div class="flex items-center justify-between">
        <div class="flex-1 flex justify-between sm:hidden">
            {{if gt .page 1}}
            <a href="#"
                class="relative inline-flex items-center px-4 py-2 border border-gray-300 text-sm font-medium rounded-md
text-gray-700 bg-white hover:bg-gray-50"
                hx-get="/web/users/by-id?page={{sub .page 1}}&limit={{.limit}}"
                hx-target="#user-container"
                hx-swap="outerHTML">
                Previous
            </a>
            {{end}}
            {{if lt .page .totalPages}}
            <a href="#"
                class="ml-3 relative inline-flex items-center px-4 py-2 border border-gray-300 text-sm font-medium rounded-
md text-gray-700 bg-white hover:bg-gray-50"
                hx-get="/web/users/by-id?page={{add .page 1}}&limit={{.limit}}"
                hx-target="#user-container"
                hx-swap="outerHTML">
                Next

```

```

        </a>
        {{end}}
    </div>
    <div class="hidden sm:flex-1 sm:flex sm:items-center sm:justify-between">
        <div>
            <p class="text-sm text-gray-700">
                Showing page <span class="font-medium">{{.page}}</span> of <span class="font-medium">
{{.totalPages}}</span>
                ({{.total}} total users)
            </p>
        </div>
        <div>
            <nav class="relative z-0 inline-flex rounded-md shadow-sm -space-x-px">
                {{if gt .page 1}}
                <a href="#"
                    class="relative inline-flex items-center px-2 py-2 rounded-l-md border border-gray-300 bg-white
text-sm font-medium text-gray-500 hover:bg-gray-50"
                    hx-get="/web/users/by-id?page={{sub .page 1}}&limit={{.limit}}"
                    hx-target="#user-container"
                    hx-swap="outerHTML">
                    Previous
                </a>
                {{end}}

                {{range $i := iterate (add $.page -2) (add $.page 3) }}
                {{if and (gt $i 0) (le $i $.totalPages)}}
                <a href="#"
                    class="relative inline-flex items-center px-4 py-2 border border-gray-300 bg-white text-sm font-
medium {{if eq $i $.page}}text-blue-600 bg-blue-50{{else}}text-gray-700 hover:bg-gray-50{{end}}"
                    hx-get="/web/users/by-id?page={{$i}}&limit={{$.limit}}"
                    hx-target="#user-container"
                    hx-swap="outerHTML">
                    {{$i}}
                </a>
            
```

```

        {{end}}
    {{end}}

    {{if lt .page .totalPages}}
    <a href="#"
      class="relative inline-flex items-center px-2 py-2 rounded-r-md border border-gray-300 bg-white
text-sm font-medium text-gray-500 hover:bg-gray-50"
      hx-get="/web/users/by-id?page={{add .page 1}}&limit={{.limit}}"
      hx-target="#user-container"
      hx-swap="outerHTML">
      Next
    </a>
    {{end}}
  </nav>
</div>
</div>
</div>
</div>
{{end}}
</div>
</div>

```

7.2. Template Features

- View toggle between "By Date" and "By ID"
- HTMX-powered pagination pointing to `/web/users/by-id`
- ID column header shows an upward arrow to indicate sort direction
- Reuses `user_row.html` partial, so CRUD actions still work
- Empty state handling



The adoc's original version omitted the search form on the by-ID page. That's intentional — there is no `SearchUsersByID`

query. If you want search here, you'd need to add a corresponding SQL query and service method. For now, this page is view-only.

8. Step 6: Add Route

Add the new route to serve the page.

8.1. Update main.go

Open `cmd/api/main.go` and add the new route:

```
webGroup := router.Group("/web")
{
    webGroup.GET("/users", webHandler.ListUsersPage)
    webGroup.GET("/users/by-id", webHandler.ListUsersByIDPage) // ← Add this line
    webGroup.GET("/users/new", webHandler.NewUserForm)
    webGroup.POST("/users", webHandler.CreateUserWeb)
    webGroup.GET("/users/:id/edit", webHandler.EditUserForm)
    webGroup.PUT("/users/:id", webHandler.UpdateUserWeb)
    webGroup.DELETE("/users/:id", webHandler.DeleteUserWeb)
    webGroup.GET("/users/stats", webHandler.GetUserStats)
}
```

8.2. Route Pattern

- `/web/users/by-id`: Main page for ID-sorted users
- Supports query parameters: `page` and `limit` for pagination
- HTMX requests receive partial template responses

8.3. Route Ordering Considerations



Gin's router prioritizes static path segments over parameterized ones, so `/users/by-id` will not be shadowed by a

hypothetical `/users/:id` route. However, it's good practice to place static routes before parameterized ones to make the intent clear and to avoid surprises if Gin's matching rules change.

9. Step 7: Update base.html for Dynamic HTMX Target

This is the most subtle part of the process. `base.html` is the full-page skeleton. It contains a `#user-container` div that HTMX populates on page load via `hx-get`. That URL must change depending on which page the user is on.

9.1. The Problem

The current `base.html` hardcodes the load endpoint:

```
<div id="user-container"
  hx-get="/web/users"
  hx-trigger="load"
  hx-target="#user-container"
  hx-swap="outerHTML"
  hx-indicator="#loading-indicator">
```

If you render `base.html` for `/web/users/by-id`, the container will still fetch `/web/users` on load — showing the wrong list. You must parameterize the endpoint.

9.2. The Fix: Pass `listEndpoint` from the handler

9.2.1. Update `ListUsersPage` full-page render

In `internal/web/web_handler.go`, in `ListUsersPage`, add `listEndpoint` to the `gin.H` map for the full-page render only (not the HTMX branch):

```
// Full page render
log.Println(" [ListUsersPage] Rendering full page: base.html")
c.HTML(http.StatusOK, "base.html", gin.H{
  "title":      "User Management",
  "users":      viewUsers,
```

```

    "page":      page,
    "limit":     limit,
    "total":     total,
    "totalPages": (total + int64(limit) - 1) / int64(limit),
    "search":    search,
    "orderBy":   "Date",
    "listEndpoint": "/web/users",    // ← ADD THIS
  })

```

9.2.2. Update base.html

In `internal/templates/layouts/base.html`, change the hardcoded URL to use the variable:

```

<div id="user-container"
  hx-get="{{.listEndpoint}}"
  hx-trigger="load"
  hx-target="#user-container"
  hx-swap="outerHTML"
  hx-indicator="#loading-indicator">
  <div id="loading-indicator" class="flex justify-center py-12 htmx-indicator">
    <div class="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
  </div>
</div>

```



The HTMX partial responses (`user_list.html` and `user_list_by_id.html`) do not need `listEndpoint`. They are swapped into `#user-container` and the container itself retains its `hx-get` attribute — but since the container is replaced with `hx-swap="outerHTML"`, the new container element must also carry a valid `hx-get`. In practice, the simplest approach is to let the partial's root `<div>` be swapped in place of the container, and have subsequent navigation happen via the pagination/toggle links inside the partial. This is how the existing `user_list.html` already works.

9.3. View Toggle in base.html

You can optionally add a view toggle to the navigation bar in `base.html`:

```
<div class="flex items-center space-x-4">
  <div class="flex gap-1 bg-gray-100 rounded-lg p-1">
    <a href="/web/users"
      class="px-3 py-1 text-sm rounded-md transition
        {{if eq .orderBy "Date"}}bg-white shadow text-gray-900{{else}}text-gray-600 hover:text-gray-900{{end}}">
      By Date
    </a>
    <a href="/web/users/by-id"
      class="px-3 py-1 text-sm rounded-md transition
        {{if eq .orderBy "ID"}}bg-white shadow text-gray-900{{else}}text-gray-600 hover:text-gray-900{{end}}">
      By ID
    </a>
  </div>
  <span id="user-count" class="text-sm text-gray-600"></span>
</div>
```



Because HTMX swaps only `#user-container` (not the whole `base.html`), the navigation toggle will not update its active class after an HTMX partial swap. The toggle links themselves are plain `<a href>` elements, so clicking them performs a full-page navigation — which does re-render `base.html` and correctly highlights the active view. This is acceptable behavior.

If you want the toggle highlight to update on partial swaps too, you'd need to either: . Include the toggle inside the partial templates (as done in Step 5), or . Use `hx-swap-oob` to update the nav separately.

For this guide, the partial-included toggle (Step 5) is the recommended approach.

10. Step 8: Test the New Page

10.1. Build and Start

```
go build ./...  
go run cmd/api/main.go
```

Or via Makefile:

```
make run
```

10.2. Test the Page

1. Navigate to <http://localhost:8080/web/users/by-id>
2. Verify users are sorted by ID (ascending)
3. Click "By Date" to switch back to the original view
4. Click "By ID" to return to the new view

10.3. Test Pagination

1. Create more than 10 users (the default page size)
2. Click through pages to verify pagination works
3. Check that sorting is maintained across pages

10.4. Test HTMX Interactions

1. Add a new user — should appear in the list
2. Edit a user — should update in place
3. Delete a user — should remove without page refresh

10.5. Verify the Endpoint

Check the browser's network tab to confirm that on page load, the `#user-container` element issues a request to `/web/users/by-id` (not `/web/users`).

11. Complete File List

Here's a summary of all files modified or created:

Table 2. Modified Files

File	Changes	Action
<code>queries.sql</code>	Added <code>ListUsersByID</code> query	Modified
<code>internal/service/user_service.go</code>	Added <code>ListUsersByID</code> method and <code>mapListUsersByIDRowToDomain</code>	Modified
<code>internal/web/web_handler.go</code>	Added <code>ListUsersByIDPage</code> handler; added <code>listEndpoint</code> to <code>ListUsersPage</code> full render	Modified
<code>cmd/api/main.go</code>	Added <code>/web/users/by-id</code> route	Modified
<code>internal/templates/layouts/base.html</code>	Parameterized <code>hx-get</code> with <code>listEndpoint</code> ; optional view toggle	Modified

Table 3. New Files

File	Description
<code>internal/templates/partials/user_list_by_id.html</code>	New partial template for ID-sorted users

12. Troubleshooting

12.1. Common Issues

Issue	Solution
cannot use ... as *repository.ListUsersRow	You reused the wrong mapper. Add <code>mapListUsersByIDRowToDomain</code> for <code>*repository.ListUsersByIDRow</code> .
function "add" not defined	Ensure template functions are registered via <code>web.GetTemplateFuncs()</code> before <code>LoadHTMLGlob</code> .
404 on <code>/web/users/by-id</code>	Check that the route is added in <code>main.go</code> and the server was restarted.
Page loads but shows date-sorted users	<code>base.html</code> still hardcodes <code>hx-get="/web/users"</code> . Pass <code>listEndpoint</code> from the handler.
Pagination links go to <code>/web/users</code>	The partial template still uses the old URL. Update all <code>hx-get</code> URLs in <code>user_list_by_id.html</code> to <code>/web/users/by-id</code> .
Toggle highlight doesn't update on partial swap	Expected. Toggle clicks perform full-page navigation. Or include the toggle inside the partial (recommended).
Users not sorted by ID	Check SQL query has <code>ORDER BY id ASC</code> clause.
HTMX not loading content	Check browser console for errors and verify <code>hx-target</code> matches element ID.

12.2. Debug Commands

```
# Check database content
psql -d MyWebApp -c "SELECT id, username, email FROM users ORDER BY id;"

# Test endpoint
curl http://localhost:8080/web/users/by-id

# Check routes are registered
```

```
grep -r "/web/users/by-id" cmd/api/
```

12.3. Logging

The handler already logs request flow. Look for these in the console:

```
[ListUsersByIDPage] Called  
[ListUsersByIDPage] Params: page=1, limit=10  
[ListUsersByIDPage] Retrieved 5 users, total: 5  
[ListUsersByIDPage] IsHTMX: true  
[ListUsersByIDPage] Rendering HTMX partial: user_list_by_id.html
```

13. Best Practices

13.1. When Adding New Pages

1. **Start with SQL:** Always add the query first and test it directly in PostgreSQL.
2. **Generate code:** Run `make generate` after adding queries.
3. **Layer by layer:** Repository → Service → Handler → Template.
4. **Add a dedicated mapper:** Don't reuse mappers across sqlc row types, even when they look identical.
5. **Parameterize the layout:** If the page uses `base.html`, make sure `listEndpoint` (or equivalent) is passed.
6. **Test each layer:** Use `make test` and manual testing.
7. **HTMX first:** Build the HTMX partial first, then the full page.

13.2. Naming Conventions

1. **Route URLs:** Use kebab-case: `/users/by-id`.
2. **Template files:** Use snake_case: `user_list_by_id.html`.
3. **Method names:** Use PascalCase: `ListUsersByIDPage`.
4. **Mapper names:** Mirror the sqlc row type: `mapListUsersByIDRowToDomain`.

13.3. Performance Considerations

1. Use pagination to limit result sets (already implemented).
2. Consider adding indexes on columns used in `ORDER BY` clauses (e.g., `CREATE INDEX idx_users_id ON users(id)`; — though the primary key already covers this).
3. Cache user counts if the database is large.

14. Conclusion

Congratulations! You've successfully added a new page to your Go + HTMX application. The process followed a clean architectural pattern:

1. **SQL Layer:** Define the query.
2. **Repository Layer:** SQLC generates type-safe code.
3. **Service Layer:** Business logic and domain mapping.
4. **Handler Layer:** HTTP request handling and HTMX detection.
5. **Template Layer:** HTML rendering with HTMX.

This pattern is consistent and can be repeated for any new page or feature you want to add to your application.

15. Additional Resources

- [HTMX Documentation](#)
- [Tailwind CSS Documentation](#)
- [Gin Framework](#)
- [sqlc Documentation](#)

16. Appendix A: Summary of Corrections from v13

This revision (v14.0) incorporates the following corrections to the original guide:

Area	Correction
Step 3	Added dedicated <code>mapListUsersByIDRowToDomain</code> . The original guide incorrectly suggested reusing <code>mapListUsersRowToDomain</code> , which would not compile because sqlc generates distinct row types per query.
Step 4	Added <code>listEndpoint</code> to the full-page render map so <code>base.html</code> can fetch the correct URL on load.
Step 5	Added view toggle inside the partial (rather than only in <code>base.html</code>) so it updates correctly on HTMX swaps.
Step 7	Added a full section explaining the <code>listEndpoint</code> parameterization — the most important fix, without which <code>/web/users/by-id</code> would silently display date-sorted users.
Step 7	Documented the toggle-highlight limitation on partial swaps, and recommended the partial-included toggle as the pragmatic solution.
Step 8	Added a verification step to check the network tab for the correct endpoint.
Troubleshooting	Added three new entries covering the mapper type mismatch, hardcoded endpoint, and toggle highlight issues.
Best Practices	Added "Add a dedicated mapper" and "Parameterize the layout" guidance.